

UNIVERSITY OF THE
WITWATERSRAND,
JOHANNESBURG



COMPUTER PROGRAMMING IN MINING
MINN2020A

Introduction to GitHub guide

Note

This is a living/dynamic/evergreen document and will constantly be updated

1. Introduction

GitHub is an online platform that brings together or allows collaboration amongst groups of individuals or teams and experts in coding. This platform is used for sharing ideas and ways of solving software-related problems. To quickly learn more about GitHub, we will look at the following questions:

a. What is GitHub?

GitHub is a code hosting platform that is used for version control and collaboration. It permits you and others to work together on projects from anywhere in the world.

b. What is Git?

Git is the actual underlying version control management software that can be installed locally by collaborators in a development team. Git facilitates the storage of code, tracking of revision history, merging of code changes, and the ability to revert to earlier code versions when required. Users may connect with the GitHub platform through git command line instructions to push or pull updates and changes.

c. What makes it beneficial?

- Firstly, it enables smooth and easy collaboration and version control thus allowing you to work on code with anyone from anywhere.
- It creates a platform for connections, learning, and portfolio aspects.
- GitHub is a robust learning and collaboration platform.
- GitHub further supports skills learning and practice in Project Management, Continuous Development Pipelines, and Agile Iterative Workflow.

2. GitHub essentials are:

• Repositories

GitHub repositories are made up of all project files and the revision history in each file. The repository can be owned individually or the ownership can be shared with other people. A development project can also be stored in a GitHub repository. The project may consist of folders and any file types for instance documents, data, images, JavaScript, CSS, HTML. Owners of a repository may also choose to store resources or any ideas they are keen to share with others on a GitHub repository. A GitHub repository typically has a

license file and a README file about the project. There are two types of repositories namely: the local repositories (Git) and remote repositories namely (GitHub).

- **Branches**

A GitHub branch is used to work with different **versions** of a repository at the same time. By default, a repository has a **master** branch (a production branch). Any other branch is a **copy** of the master branch (as it was at a point in time). New Branches are for bug fixes and feature work separate from the master branch. When changes are ready, they can be merged into the master branch. If you make changes to the master branch while working on a new branch, these updates can be pulled in.

- **Commits**

At GitHub, saved and logged changes are called commits. Each commit (change) has a description explaining why a change was made. Commits are similar to bookmarks; these are screenshots of the source code at a given time. One main advantage is that collaborators may revert to previous commits, which allows for iterative workflows and creating back-up points in complex projects. Each commit has a unique identifying ID (hash key) making it simpler to reference previous work.

- **Pull Requests**

Pull Requests are the heart of GitHub **collaboration**. With a pull request, you are **proposing** that your changes should be **merged** (pulled in) with the master. Pull requests show content **differences**, changes, additions, and subtractions in **colors** (green and red). As soon as you have a commit, you can open a pull request and start a discussion, even before the code is finished.

- **Git**

Git is the version control software upon which GitHub is built.

3. How do I get started on GitHub?

1. Sign up for GitHub

2. Install Git
3. Create a Repository
4. Run Git Bash
5. Create a Branch
6. Create and Commit Changes to a Branch
7. Open a Pull Request
8. Merge Your Pull Request

The details of the steps (1 to 8) provided above will now be discussed.

Step 1: Sign up for GitHub

In order to use GitHub, you'll need a GitHub account. You can create a free GitHub account by following these steps:

1. click or copy and paste the link provided below into a browser. You could also type out the link in a browser. <https://github.com/>

You will be directed to the GitHub landing page shown in Figure 1

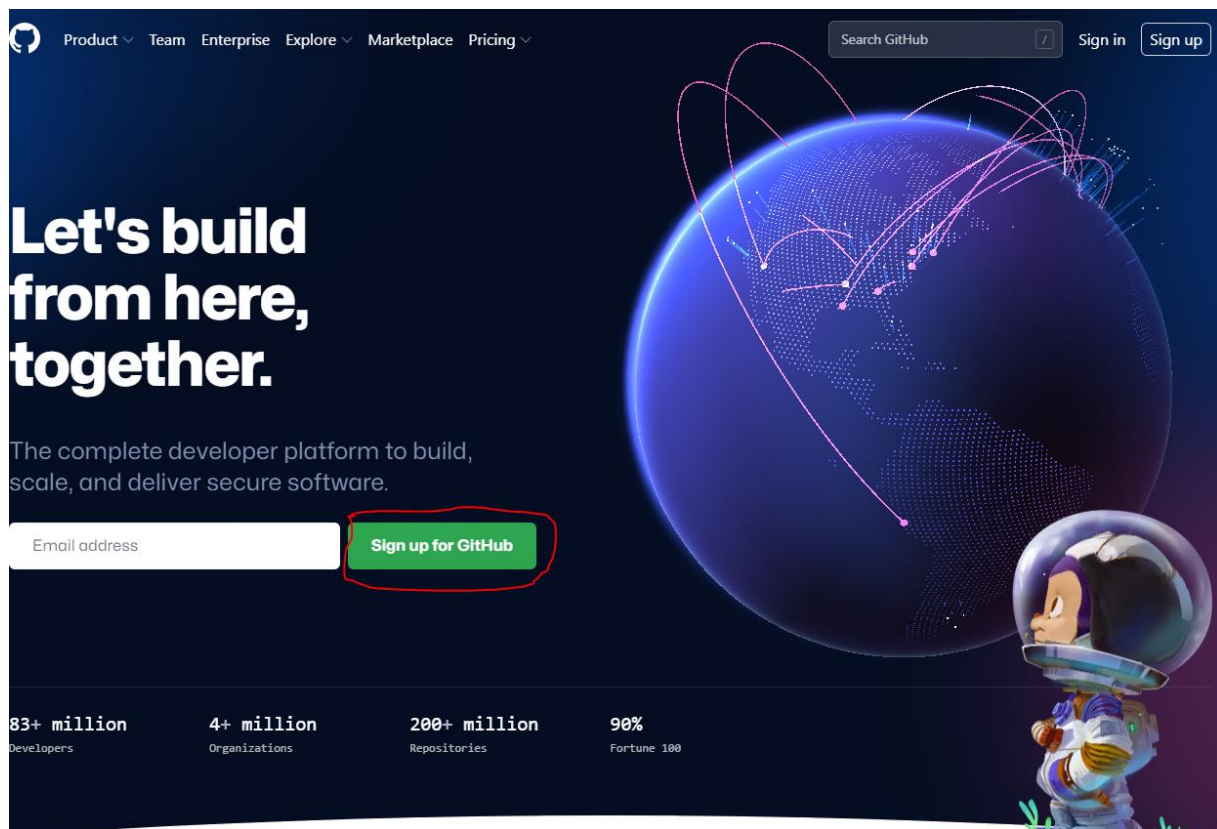


Figure 1 GitHub landing Page

2. Enter your email address and click on sign up for GitHub (See Figure 1, circled in red)
3. Another prompt will appear as shown in Figure 2

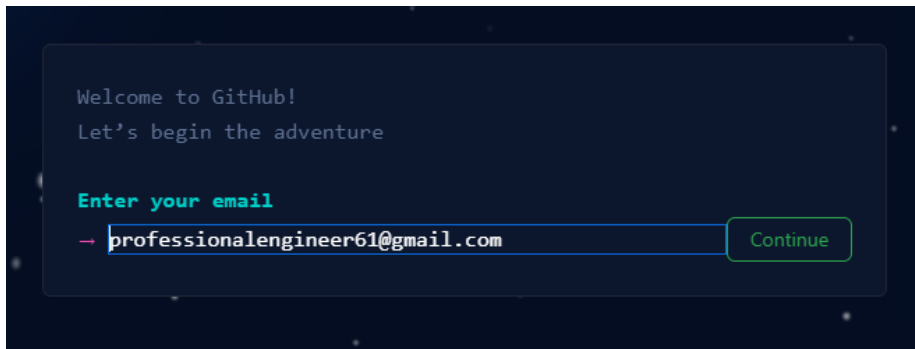


Figure 2: Prompt box for GitHub sign in

4. Click on continue
5. Another Prompt box showing your email has been verified and now your password is required will appear on the screen as shown

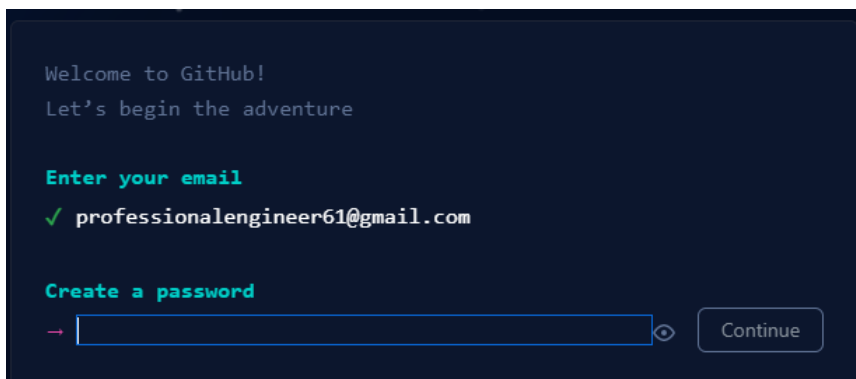


Figure 3: Creating a password

6. Enter a "strong password and click continue"
Hints: A strong password may consist of a word with more than 8 letters, starting with an Uppercase letter, then low case letters, followed by a special character
7. The next prompt that appears will request for your User Name

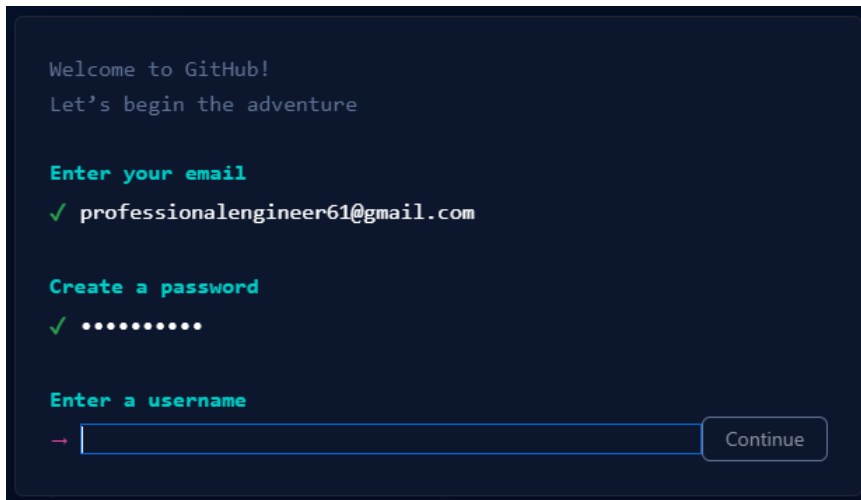


Figure 4: Creating a user name

8. Get a valid and available user name as shown in figure 5.

Hint: Try out a variety of user names that are close to your name should you receive a message indicating that your user name is not available.

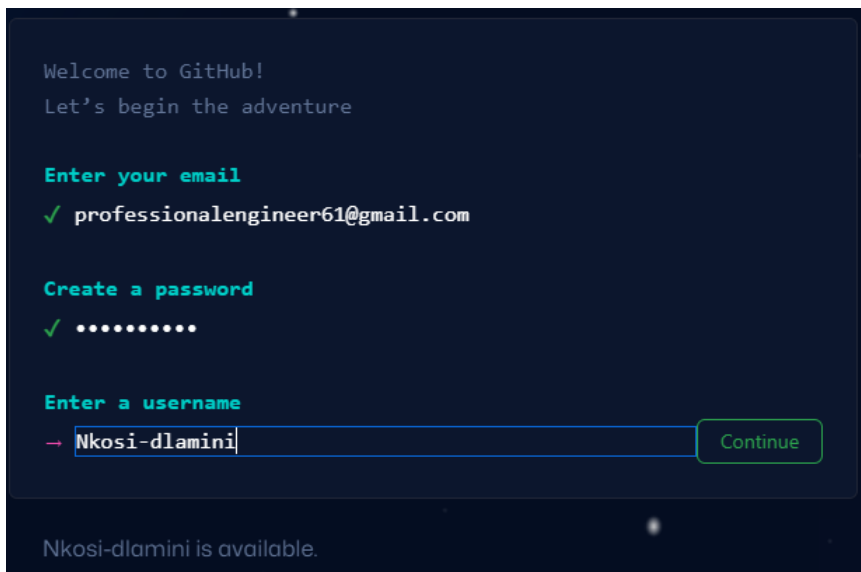
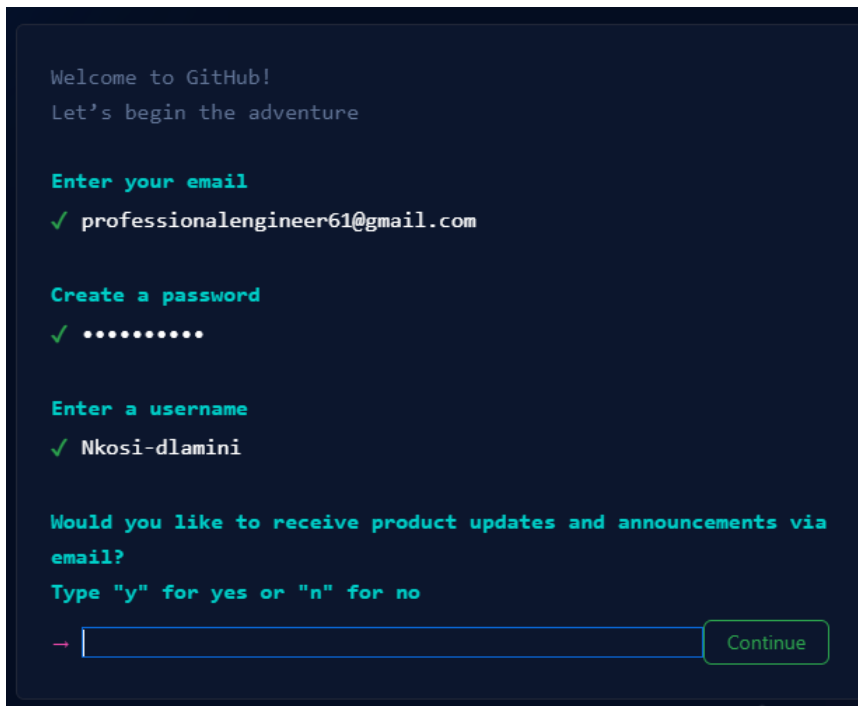


Figure 5: Valid user name

9. Once a prompt indicating the user name is available shows, click on continue

10. A prompt asking if you wish to receive product updates will appear as shown in figure 6, click “y” if you would like to receive updates or “n” if you do not wish to receive updates. Then click on continue



```

Welcome to GitHub!
Let's begin the adventure

Enter your email
✓ professionalengineer61@gmail.com

Create a password
✓ .....

Enter a username
✓ Nkosi-dlamini

Would you like to receive product updates and announcements via
email?
Type "y" for yes or "n" for no
→  Continue

```

Figure 6: Product updates

11. You now have to verify your account as shown in figure 7, click on start puzzle to continue

Let's begin the adventure

Enter your email

✓ professionalengineer61@gmail.com

Create a password

✓

Enter a username

✓ Nkosi-dlamini

Would you like to receive product updates and announcements via email?

Type "y" for yes or "n" for no

✓ "y"

Verify your account

Please solve this puzzle to verify that you are human

Click "Start puzzle" to continue

Start puzzle

Figure 7: Account verification

12. The puzzle will appear as shown in figure 8 and you have to follow the prompts as they appear on the screen



Figure 8: Further account verification

13. You will receive another prompt on the screen indicating “Working please wait” meaning the account is being set up. Please be patient with this stage, it might take a few minutes. If it takes more than 5 minutes, then replay the puzzle and carefully choose the correct answers

Note: Some individuals might not experience this stage. Please proceed to the next stage.

Let's begin the adventure

Enter your email

✓ professionalengineer61@gmail.com

Create a password

✓

Enter a username

✓ Nkosi-dlamini

Would you like to receive product updates and announcements via email?

Type "y" for yes or "n" for no

✓ "y"

Verify your account

Working, please wait...



Figure 9: Account setup

14. Once the correct answer has been chosen, a verification tick will appear where the puzzle was.
Click on create account

```
Enter a username
✓ Nkosi-dlamini

Would you like to receive product updates and announcements via
email?
Type "y" for yes or "n" for no
✓ "y"

Verify your account

[Large empty box with a green checkmark in the center]

Create account
```

Figure 10: completed account verification

15. Verification code is then sent to your email. Check your email for the code and enter the code in the prompt box

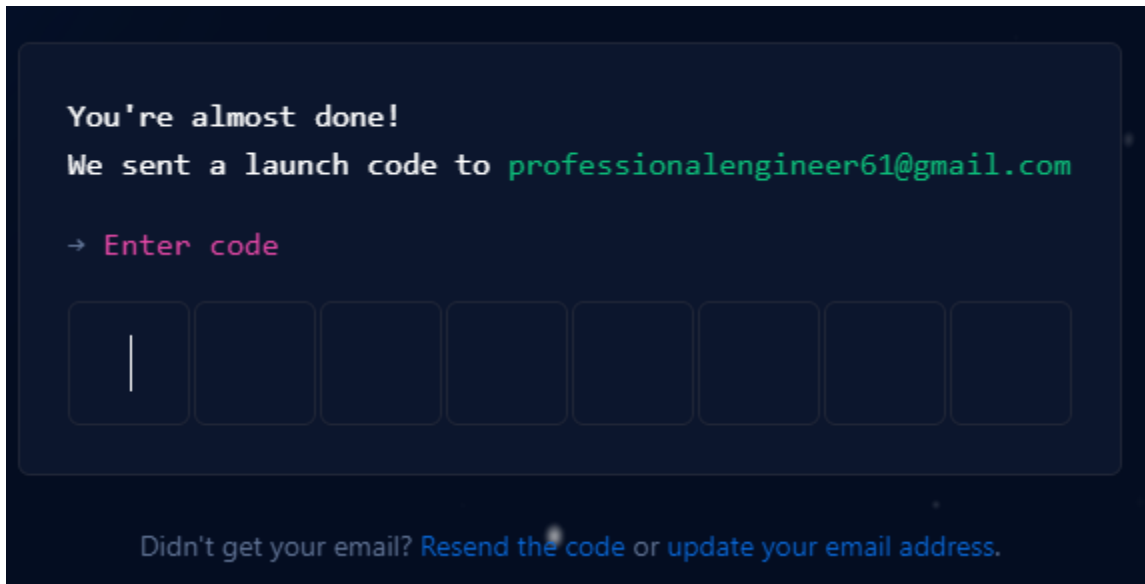


Figure 11: Required code emailed to the user

16. After entering the code obtained from your email, a personalization screen will appear as shown. Indicate that you are a student and choose 2-5 team members and also chose student and click continue

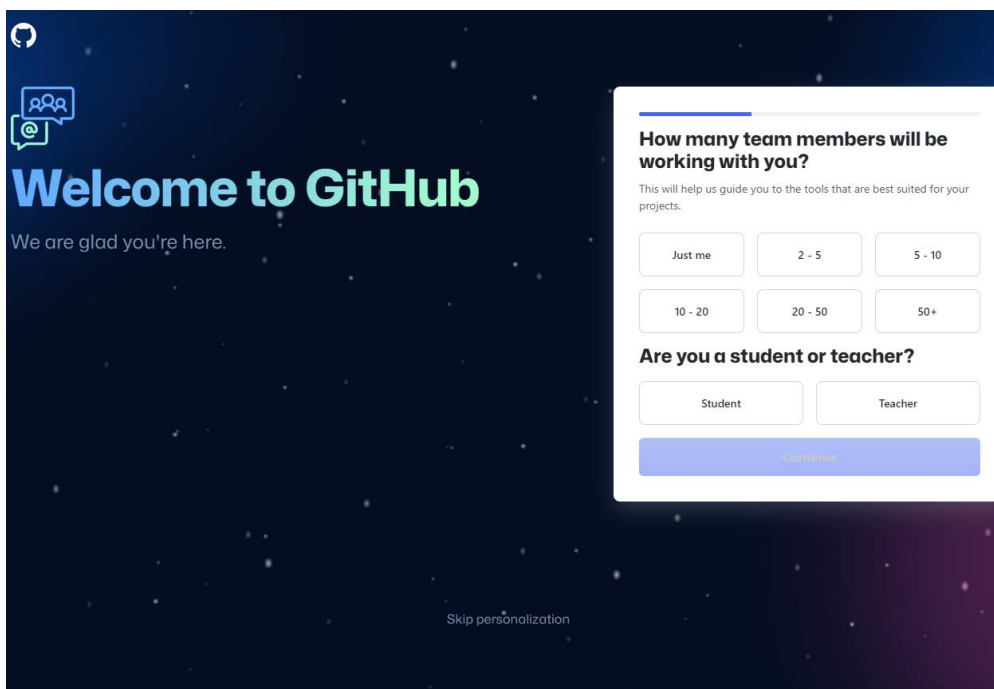


Figure 12: GitHub Personalization

17. Click on the tools required. Kindly choose all the tools and click continue

The tools you need to build what you want.

Soup to nuts, GitHub has it all.

Select all that apply so we can point you to the right GitHub plan.

- ☐  Collaborative coding
Codespaces, Pull requests, Notifications, Code review, Code review assignments, Code owners, Draft pull requests, Protected branches, and more.
- ☐  Automation and CI/CD
Actions, Packages, APIs, GitHub Pages, GitHub Marketplace, Webhooks, Hosted runners, Self-hosted runners, Secrets management, and more.
- ☐  Security
Private repos, 2FA, Required reviews, Required status checks, Code scanning, Secret scanning, Dependency graph, Dependabot alerts, and more.
- ☐  Client Apps
GitHub Mobile, GitHub CLI, and GitHub Desktop.
- ☐  Project Management
Projects, Labels, Milestones, Issues, Unified Contribution Graph, Org activity graph, Org dependency insights, Repo insights, Wikis, and GitHub Insights.
- ☐  Team Administration
Organizations, Invitations, Team sync, Custom roles, Domain verification, Audit Log API, Repo creation restriction, and Notification restriction.
- ☐  Community
GitHub Marketplace, GitHub Sponsors, GitHub Skills, Electron, and Atom.

Figure 13: Choice of required tools

18. Your packages will now show, click on continue for free.

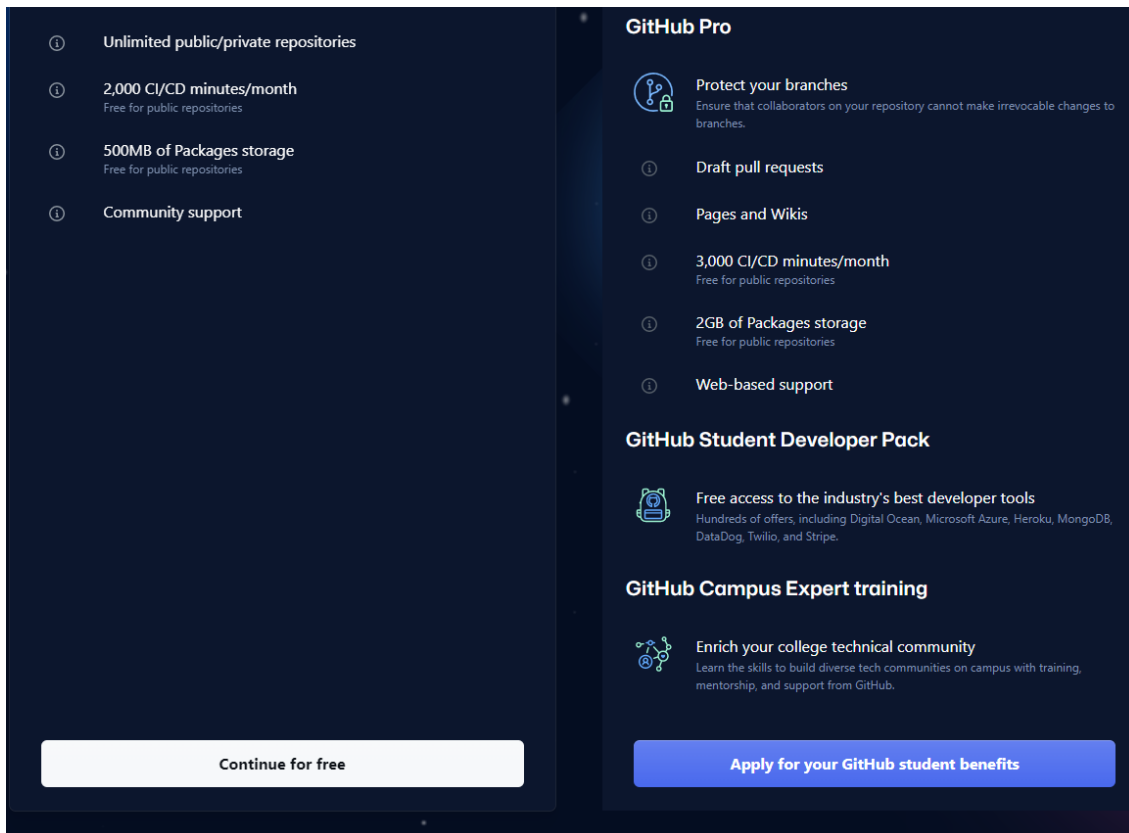


Figure 14: Choosing Github packages

19. Congratulations! You finally have a Github account.

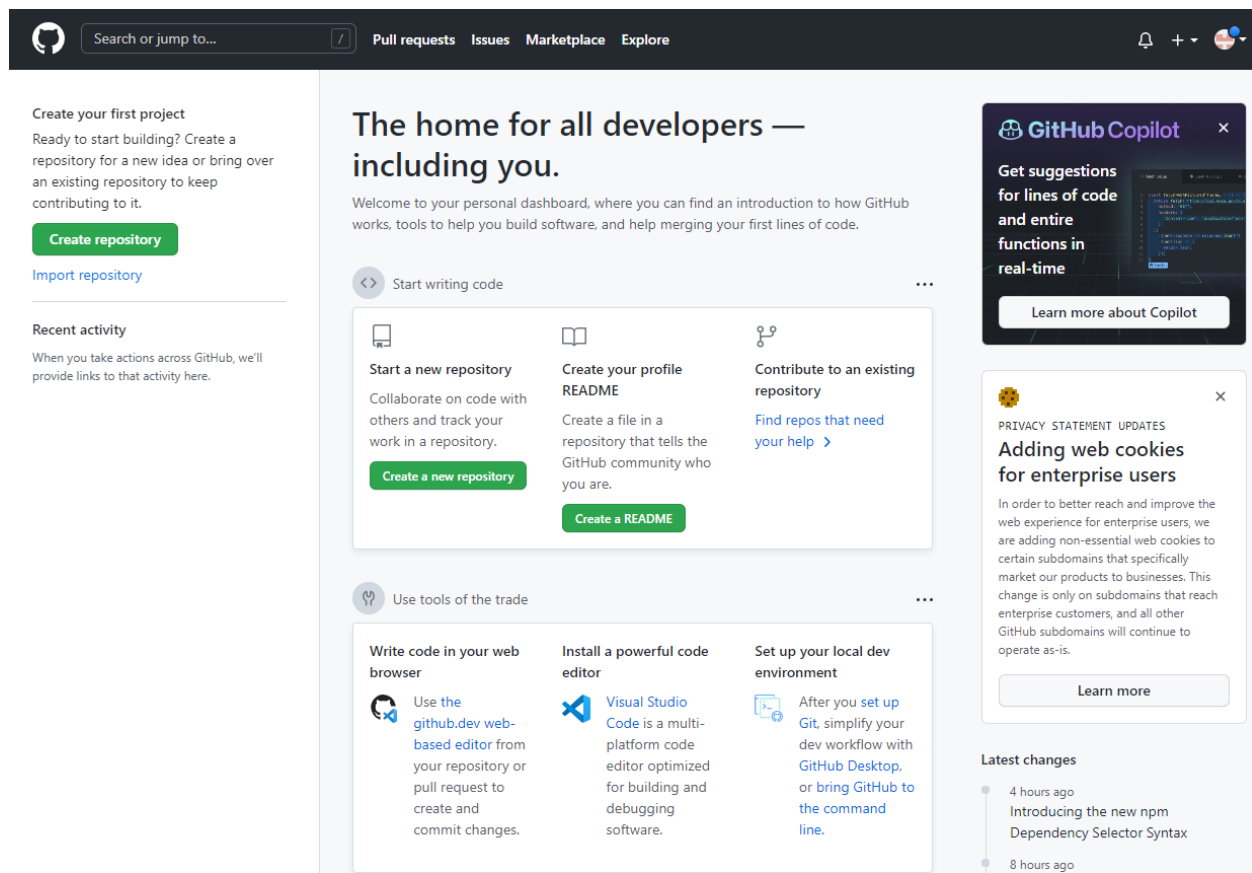


Figure 15: The GitHub account

With a free account, you'll get access to unlimited public and private repositories. You'll also get bug tracking and project management features.

Step 2: Install Git

GitHub runs on Git. Git is a version-control system created by programming icon, Linus Torvald. He originally created Git to track changes to source code changes as he was developing the Linux operating system. Git helps programmers collaborate, coordinate work, and work together on complex code and development projects. Git tracks changes and helps teams work remotely on intricate programs. Copy and paste the link provided in a browser or you can type it as well.

<https://git-scm.com/download/win>

Download for Windows

[Click here to download](#) the latest (2.37.1) 64-bit version of **Git for Windows**. This is the most recent [maintained build](#). It was released **22 days ago**, on 2022-07-12.

Other Git for Windows downloads

Standalone Installer

[32-bit Git for Windows Setup](#).

[64-bit Git for Windows Setup](#).

Portable ("thumbdrive edition")

[32-bit Git for Windows Portable](#).

[64-bit Git for Windows Portable](#).

Using winget tool

Install [winget tool](#) if you don't already have it, then type this command in command prompt or Powershell.

```
winget install --id Git.Git -e --source winget
```

The current source code release is version **2.37.1**. If you want the newer version, you can build it from [the source code](#).

Figure 1: Git Installation Page

If you are using the Linux operating system, the following link can be used:

<https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>

Step 3: Create a Repository

A repository (or repo) is essentially synonymous with the word “project.” Quite simply, a repository stores everything pertinent to a specific project including files, images, spreadsheets, data sets, and videos, often sorted into files. It’s best to include a README file within your repository that has specific information regarding the given project. On GitHub, you can add a README file right when you create a new repository. To create a new repository, you’ll hit “new repository” in the upper-right-hand corner of your homepage after you have created your free account. You can then name your repository, include a brief description, and check the box that says “initialize this repository with a README.” Finally, you’ll click “create a repository.”



The screenshot shows the 'Create a new repository' page on GitHub. At the top, it says 'Create a new repository' and provides a brief explanation: 'A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)'. Below this, there are two input fields: 'Owner' with a dropdown menu showing 'PrinJonas' and 'Repository name' with a text input 'Software-Test-Repo.' and a green checkmark. A note below these fields states: 'Great repository names are short and memorable. Need inspiration? How about [probable-guide?](#)'. The 'Description' section is optional and has a text input with the text 'This is a test repository that demonstrates how to Push and Pull from a local repository.' Below the description, there are two radio button options: 'Public' (selected) and 'Private'. The 'Public' option has a subtext: 'Anyone on the internet can see this repository. You choose who can commit.' The 'Private' option has a subtext: 'You choose who can see and commit to this repository.' The 'Initialize this repository with:' section has a subtext: 'Skip this step if you're importing an existing repository.' It includes a checked checkbox for 'Add a README file' with a subtext: 'This is where you can write a long description for your project. [Learn more.](#)'. Below this is the 'Add .gitignore' section with a subtext: 'Choose which files not to track from a list of templates. [Learn more.](#)' and a dropdown menu showing '.gitignore template: None'. The 'Choose a license' section has a subtext: 'A license tells others what they can and can't do with your code. [Learn more.](#)' and a dropdown menu showing 'License: None'. At the bottom, it says 'This will set `main` as the default branch. Change the default name in your [settings](#).' and a note: 'You are creating a public repository in your personal account.'

Figure 2: Creating a Repository on GitHub

Step 4: Run Git Bash

After having installed Git, create and open a new folder on your computer. Right click anywhere in that folder and then select the Git Bash Here option. That will open the bash terminal as shown.

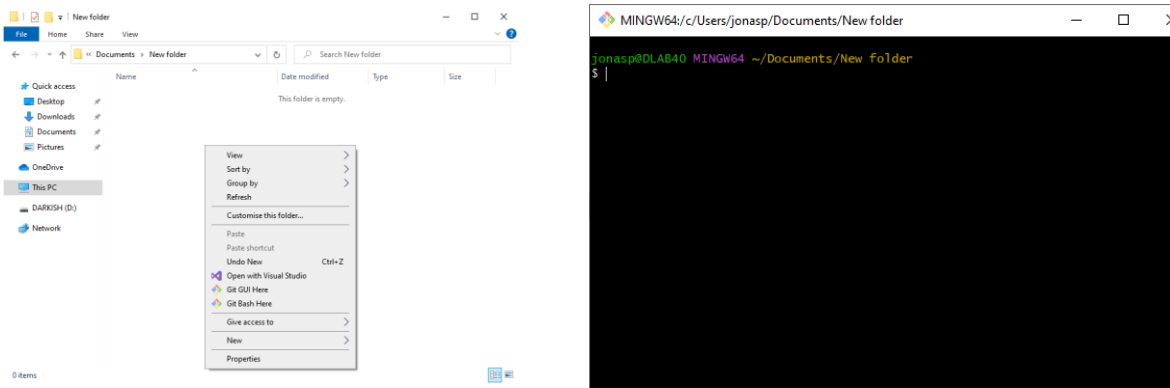


Figure 3: Running Git Bash

To make the connection between Git (version control) and GitHub (online platform), you may perform a process known as cloning; this is in essence creating a local version of your newly created GitHub repository. The illustrations below show the two steps: firstly, copy the cloning HTTPS link by clicking the Code button on your repository and secondly, use the git clone command on Git Bash as shown.

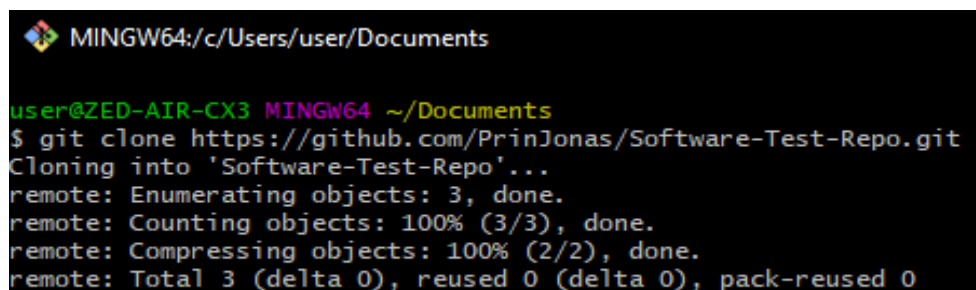
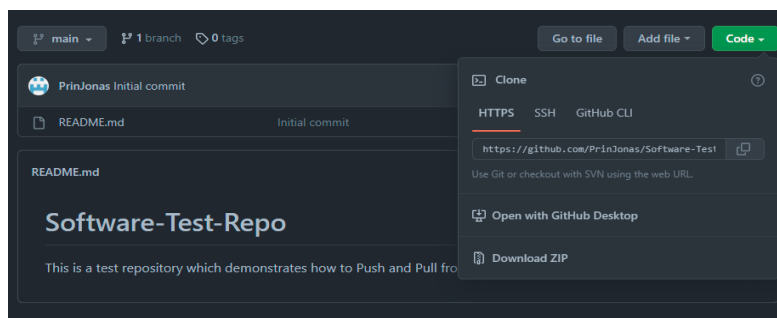


Figure 4: Cloning a Repository from GitHub

Note, the local version of the repository needs to be updated frequently such that it does not trail behind the shared repository on the GitHub platform

Step 5: Create a Branch

Projects are multi-faceted and many program versions are required when you're building. Branching enables you to edit multiple unique versions of a repository at once. Your repository automatically has a definitive branch called **master**. You can work on several different branches to make edits before eventually committing them to the master branch. When a new branch is started, it'll be a copy of the master branch until you edit it to make new changes. A branch typically goes through many steps and approvals before it is ever merged into the master branch.

To start a new branch in GitHub, navigate to your new repository, click the dropdown that reads "branch: master," type a branch name (like README-edits), and then hit "create branch." Branches are ideal for new features or bug fixes. Likewise, there are powerful git commands that may be used to achieve branching of source code versions. Once a repository has been cloned, The illustration below shows the list of commands utilized to create branches that run parallel to the master/main branch.

```
user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git branch
* main

user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git branch FirstBranch

user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git branch SecondBranch

user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git checkout FirstBranch
Switched to branch 'FirstBranch'

user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (FirstBranch)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

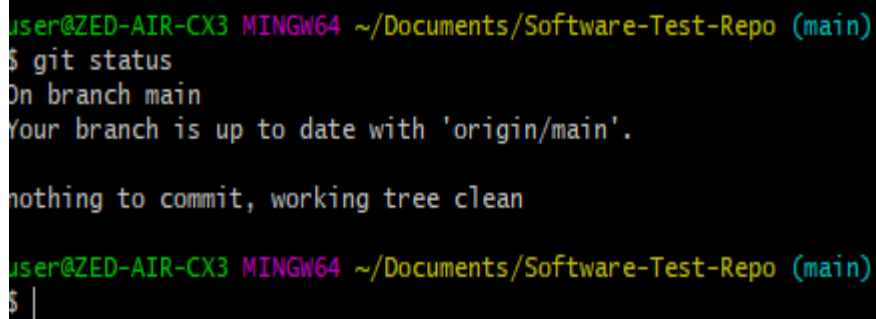
Figure 5: Creating Branches with Git

Step 6: Create and Commit Changes to a Branch

To make changes to a branch in GitHub, go to the code view for your newly created branch. Click the file you want to change, then hit the pencil icon in the upper right, make any necessary edits, describe

your changes by writing a commit message, and then click “commit changes.” Each saved change is called a commit. Every individual commit has its own commit message which gives more details into why a specific change was done. The commit messages give a history of changes and help project contributors understand how the project has changed over time.

When utilizing git, the process of bookmarking or creating commits of the source code may be categorized into two parts: *the tracking of changes in files* and *the staging of modified files* for committing. From the figure 7 below, the git status command shows that there is nothing to track since no modifications have been made. Figure 8 however shows that new additions have been made and the staging process may be implemented. To do this, the add command is used thereafter a commit may be made for these changes. Note that by using the “-m” tag on the commit instruction, a custom message may be entered to easily identify what the commit is about similar to the GitHub procedure.

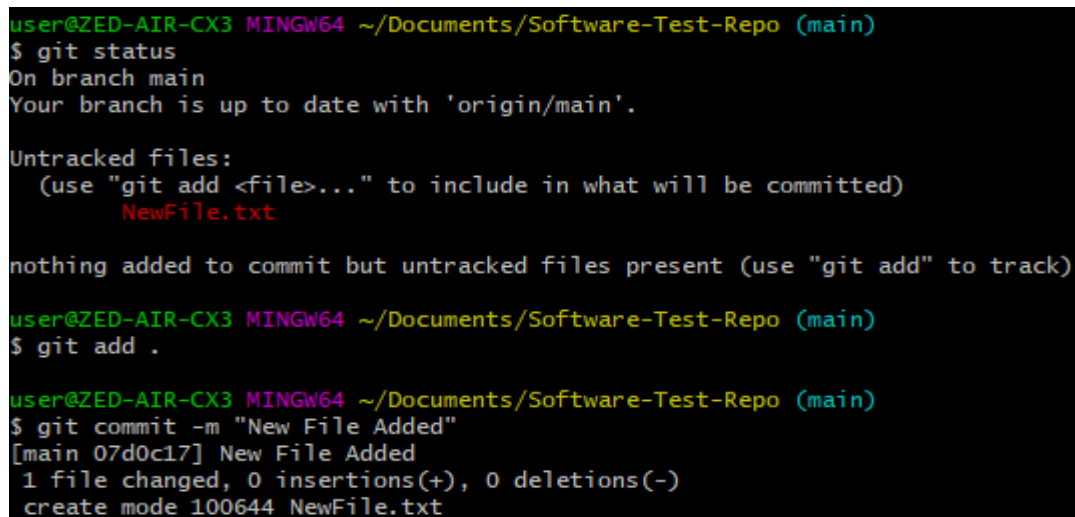


```
user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ |
```

Figure 6: Git with no new changes



```
user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    NewFile.txt

nothing added to commit but untracked files present (use "git add" to track)

user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git add .

user@ZED-AIR-CX3 MINGW64 ~/Documents/Software-Test-Repo (main)
$ git commit -m "New File Added"
[main 07d0c17] New File Added
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 NewFile.txt
```

Figure 7: Git with new changes

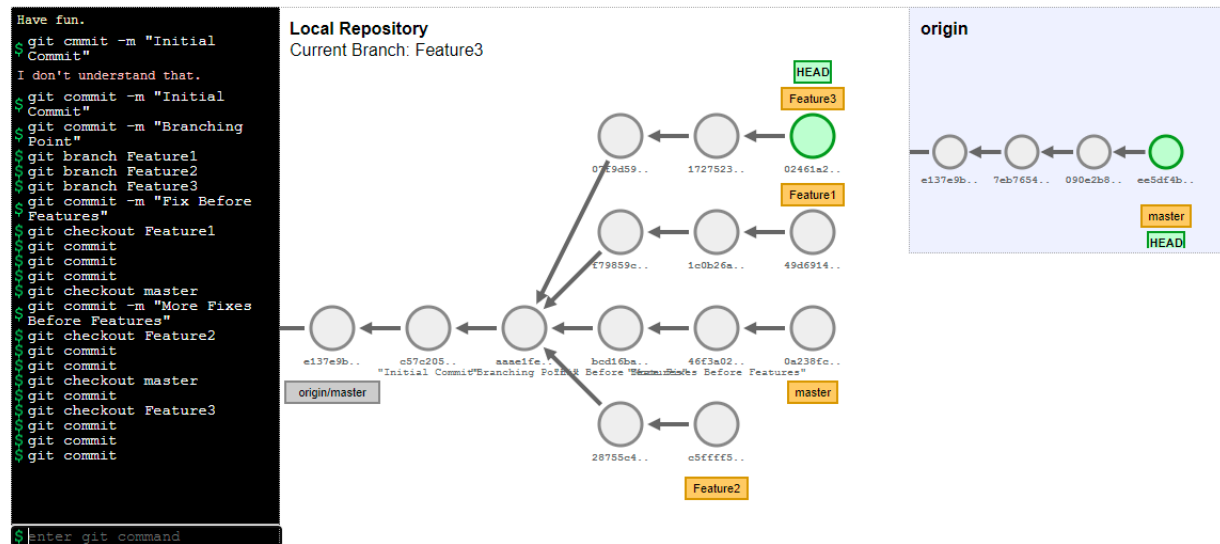


Figure 8: General view of commit and branching history

From a bird's eye view, the illustration above gives a general idea of how a commit and branching history for a typical project may look like. Note that multiple commits have been made, each having a unique ID. A visual representation of how branching works is shown by the Feature branches created. The "head" element is always the position where your version control was last updated.

Step 7: Open a Pull Request

In order for any branch to be merged into another person's branch, you must open a pull request. A pull request is GitHub's way of notifying relevant parties about your request to incorporate changes into their branch. A pull request will show in red and green the differences of the content between branches. You can make a pull request any time you complete a commit. For best results, when sending a pull request, you can use the "@" feature to mention specific people from whom you need feedback.

To open a pull request, you'll go to the "pull request" tab and hit the button that says "new pull request." Next, in the "example comparisons" box, find the branch you made and compare it with the master. Ensure you like the changes and then click the "Create pull request" button. Title your pull request and briefly describe the changes. To finish, click "Create pull request."

Step 8: Merge Your Pull Request

Merging your pull request with the master branch is something you may need to pass on to your superiors to handle. For the sake of learning, though, you can practice doing it yourself. Simply hit the button that says “merge pull request,” select “confirm merge,” and then delete the branch you merged once it has been incorporated into the master.

Conclusion

A brief introduction to GitHub has been presented. The signing up for a GitHub account, creating repositories, branches, commits and pull requests has also been discussed. It is anticipated that a user who read the quick introduction and followed the steps is now more familiarized with the use of GitHub.

Sources of information

1. GitHub [last accessed 3rd August 2022], <https://github.com/>

Appendix A

This appendix provides extra material that a student can use to further their understanding of GitHub

Extra Material

Videos

<https://www.youtube.com/watch?v=w3jLJU7DT5E>

<https://www.youtube.com/watch?v=8Dd7KRpKeaE>